

Single-Cycle 32-bit RISC-V CPU

1. Introduction

This Project implements a single-cycle RISC-V processor conforming to the RV32I base integer instruction set architecture. The processor is written entirely in Verilog HDL and is focused on clarity and correctness rather than performance optimization. The goal is to design, implement, and verify a functional CPU capable of executing arithmetic, logical, branch, jump and memory operations using a clean and understandable datapath.

The design was verified through extensive simulation using hand-written test programs covering arithmetic, logic, control flow, and memory access instructions.

2. Overall Architecture

The processor follows a single-cycle datapath architecture, meaning that all five classic stages of instruction execution occur in one clock cycle:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

Datapath Characteristics:

- No pipeline registers are used
- All combinational logic settles within one cycle
- Control signals are generated purely from the current instruction
- Memory and register writes occur on the rising clock edge

This approach was taken to simplify debugging and verification. It also exposes the relationship between instructions and hardware behavior.

Main Components:

- Program Counter (PC)
- Instruction Memory (ROM)
- Control Unit
- Register File
- Immediate Generator
- Arithmetic Logic Unit (ALU)
- Data Memory
- Branch & Jump Logic

Each component is implemented as a separate Verilog module and connected at the top level.

3. Instruction Fetch

The Program Counter (PC) holds the address of the current instruction. On each cycle:

- Normally, $PC = PC + 4$
- If a branch is taken, $PC = PC + \text{branch_offset}$

- If a jump (JAL/JALR) occurs, PC is updated accordingly

The instruction memory outputs a 32-bit instruction based on the current PC value.

4. Instruction Decode & Control Unit

The control unit decodes three main instruction fields:

Field	Bits	Purpose
opcode	[6:0]	Instruction category
funct3	[14:12]	Operation subtype
funct7	[31:25]	Extended operation selector

Table 1.

Based on these fields, it generates control signals:

- reg_write
- mem_read, mem_write
- mem_to_reg
- alu_op
- alu_src
- branch, branch_type
- jal, jalr
- is_lui, is_auiipc
- load_type

This design supports the following instruction categories:

- R-type ALU operations
- I-type ALU operations
- Load instructions (LB, LH, LW, LBU, LHU)
- Store instructions (SW)
- Branch instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU)
- Jump instructions (JAL, JALR)
- Upper immediate instructions (LUI, AUIPC)

Supported opcodes and meanings:

Opcode	Binary	Instruction Type
0110011	0x33	R-type (register ALU)
0010011	0x13	I-type ALU
0000011	0x03	Loads
0100011	0x23	Stores
1100011	0x63	Branches
1101111	0x6F	JAL
1100111	0x67	JALR
0110111	0x37	LUI
0010111	0x17	AUIPC

Table 2.

5. Immediate Generator

The immediate generator extracts and sign-extends immediates based on instruction format. This allows correct address calculation for loads, stores, branches, and jumps.

Immediate types:

Type	Used By	Example
I-type	ADDI, LW, JALR	addi x1, x0, 5
S-type	SW	sw x3, 0(x2)
B-type	BEQ, BLT	beq x1, x2, label
U-type	LUI, AUIPC	lui x1, 0x12345
J-type	JAL	jal x1, label

Table 3.

6. Register File

The register file contains 32 registers (x0-x31). Two read ports, one write port.

Key properties:

- x0 always reads as zero
- Reads are combinational
- Writes occur on clock rising edge
- Write is ignored if destination register is x0

Example: addi x5, x0, 10. Reads x0 = 0, writes 10 to x5.

7. Arithmetic Logic Unit (ALU)

The ALU supports all RV32I arithmetic and logical operations. Additionally, the ALU provides comparison signals:

- eq: equality check
- lt_signed: signed less-than

Unsigned comparisons for branches use direct unsigned comparisons outside the ALU.

Supported ALU operations:

Operation	Description
ADD	Addition
SUB	Subtraction
AND	Bitwise AND
OR	Bitwise OR
XOR	Bitwise XOR
SLL	Shift left logical
SRL	Shift right logical
SRA	Shift right arithmetic
SLT	Signed less-than
SLTU	Unsigned less-than

Table 4.

Example: slt x3, x1, x2. Writes 1 if $x1 < x2$ (signed). Otherwise writes 0.

8. Branch & Jump Logic

Branch decisions use ALU comparison outputs. They are made using:

- eq
- lt_signed
- unsigned comparisons

Branch conditions:

Instruction	Condition
BEQ	rs1 == rs2
BNE	rs1 != rs2
BLT	signed <
BGE	signed >=
BLTU	unsigned <
BGEU	unsigned >=

Table 5.

Example: bltu x1, x2, label. Uses unsigned comparison. Useful for memory addresses and bit patterns.

9. Data Memory

The memory is byte-addressable and little-endian. It is internally stored as reg [7:0] mem[].

Supported operations:

- SW (store word)
- LW (load word)
- LB/LBU (load byte signed/unsigned)
- LH/LHU (load halfword signed/unsigned)

Memory layout example:

Storing 0x87654321 at address 8:

Address	Value
8	0x21
9	0x43
10	0x65
11	0x87

Table 6.

Load examples:

- lb x10, 8(x2) // sign-extends 0x21
- lbu x11, 8(x2) // zero-extends 0x21
- lh x12, 8(x2) // sign-extends 0x4321
- lhu x13, 8(x2) // zero-extends 0x4321

Only valid aligned addresses are assumed, simplifying the design while remaining compliant the base ISA expectations.

10. Write-Back Stage

The value written back to the register file is selected from:

- ALU result
- Memory read data
- PC + 4 (for JAL/JALR)
- Immediate value (for LUI)
- PC + immediate (for AUIPC)

The mem_to_reg and instruction-specific control signals select the correct source.

Example: jal x1, label. $x1 = PC + 4$

11.Verification & Testing

Multiple test programs were written to verify:

- Immediate ALU operations
- Register-register ALU operations
- Signed and unsigned comparisons
- Branch correctness
- Jump behavior
- Load/store correctness
- Byte and halfword memory
- Edge cases (negative values, large shifts)

Testbench instructions for testing:

// Immediate ALU instructions

```
instr_rom[0] = 32'h00500093; // addi x1, x0, 5
instr_rom[1] = 32'h00A00113; // addi x2, x0, 10
instr_rom[2] = 32'h00308193; // addi x3, x1, 3 (x3 = 8)
instr_rom[3] = 32'h0020F213; // andi x4, x1, 2 (5 & 2 = 0)
instr_rom[4] = 32'h0020E293; // ori x5, x1, 2 (5 | 2 = 7)
instr_rom[5] = 32'h0030C313; // xori x6, x1, 3 (5 ^ 3 = 6)
instr_rom[6] = 32'h00109193; // slli x3, x1, 1 (5 << 1 = 10)
instr_rom[7] = 32'h00115213; // srli x4, x2, 1 (10 >> 1 = 5)
instr_rom[8] = 32'h40115293; // srai x5, x2, 1 (arithmetic shift)
instr_rom[9] = 32'h0020A313; // slti x6, x1, 2 (5 < 2 ? 0)
instr_rom[10] = 32'h0060B393; // sltiu x7, x1, 6 (5 < 6 ? 1)
```

// Register-register ALU ops

```
instr_rom[11] = 32'h002081B3; // add x3, x1, x2 (5 + 10 = 15)
instr_rom[12] = 32'h40208233; // sub x4, x1, x2 (5 - 10 = -5)
instr_rom[13] = 32'h0020F2B3; // and x5, x1, x2
instr_rom[14] = 32'h0020E333; // or x6, x1, x2
instr_rom[15] = 32'h0020C3B3; // xor x7, x1, x2
instr_rom[16] = 32'h00209433; // sll x8, x1, x2[4:0]
instr_rom[17] = 32'h0020D4B3; // srl x9, x1, x2
instr_rom[18] = 32'h4020D533; // sra x10, x1, x2
instr_rom[19] = 32'h0020A5B3; // slt x11, x1, x2
instr_rom[20] = 32'h0020B633; // sltu x12, x1, x2
```

// Memory operations

instr_rom[21] = 32'h00312023; // sw x3, 0(x2) (store 15)

instr_rom[22] = 32'h00012183; // lw x3, 0(x2) (load back 15)

// Branch instructions

instr_rom[23] = 32'h00208463; // beq x1, x2, +8 (not taken)

instr_rom[24] = 32'h00100193; // addi x3, x0, 1 (executed)

instr_rom[25] = 32'h00109463; // bne x1, x1, +8 (not taken)

instr_rom[26] = 32'h00200193; // addi x3, x0, 2

instr_rom[27] = 32'h0020C463; // blt x1, x2, +8 (taken)

instr_rom[28] = 32'h00300193; // addi x3, x0, 3 (skipped)

instr_rom[29] = 32'h0020D463; // bge x2, x1, +8 (taken)

instr_rom[30] = 32'h00400193; // addi x3, x0, 4 (skipped)

//BLTU

instr_rom[31] = 32'hFFF00093; // addi x1, x0, -1 (x1 = 0xFFFFFFFF)

instr_rom[32] = 32'h00100113; // addi x2, x0, 1

// BLTU should NOT be taken (unsigned: 0xFFFFFFFF > 1)

instr_rom[33] = 32'h0020E463; // bltu x1, x2, +8

instr_rom[34] = 32'h00100293; // addi x5, x0, 1

instr_rom[35] = 32'h00200293; // addi x5, x0, 2

// LUI

instr_rom[36] = 32'h123450B7; // lui x1, 0x12345

// Jumps

instr_rom[37] = 32'h004000EF; // jal x1, +4

instr_rom[38] = 32'h00500113; // addi x2, x0, 5

// AUIPC

instr_rom[39] = 32'h00001297; // auipc x5, 0x1 (x5 = pc + 0x1000)

// Byte/Half loads

instr_rom[40] = 32'h00010283; // lb x5, 0(x2)

instr_rom[41] = 32'h00014303; // lbu x6, 0(x2)

instr_rom[42] = 32'h00011383; // lh x7, 0(x2)

instr_rom[43] = 32'h00015403; // lhu x8, 0(x2)

// Testing Loads/Stores with larger values

instr_rom[44] = 32'h87654337; // lui x6, 0x87654

instr_rom[45] = 32'h32130313; // addi x6, x6, 0x321 (x6 = 0x87654321)

```

instr_rom[46] = 32'h00612423; // sw x6, 8(x2) (store 0x87654321 at address 18)
instr_rom[47] = 32'h00814503; // lb x10, 8(x2) (load byte = 0x21 = 33 decimal)
instr_rom[48] = 32'h00814583; // lbu x11, 8(x2) (load unsigned byte = 0x21)
instr_rom[49] = 32'h00811603; // lh x12, 8(x2) (load halfword = 0x4321)
instr_rom[50] = 32'h00815683; // lhu x13, 8(x2) (load unsigned halfword = 0x4321)
// End (infinite loop)

instr_rom[51] = 32'h0000006F; // jal x0, 0 (halt)

```

Simulation results:

```

# Time=50000 | PC=00000000 | INSTR=00500093 | ALU=00000005 | WB=00000005
# Time=70000 | PC=00000004 | INSTR=00a00113 | ALU=0000000a | WB=0000000a
# Time=90000 | PC=00000008 | INSTR=00308193 | ALU=00000008 | WB=00000008
# Time=110000 | PC=0000000c | INSTR=0020f213 | ALU=00000000 | WB=00000000
# Time=130000 | PC=00000010 | INSTR=0020e293 | ALU=00000007 | WB=00000007
# Time=150000 | PC=00000014 | INSTR=0030c313 | ALU=00000006 | WB=00000006
# Time=170000 | PC=00000018 | INSTR=00109193 | ALU=0000000a | WB=0000000a
# Time=190000 | PC=0000001c | INSTR=00115213 | ALU=00000005 | WB=00000005
# Time=210000 | PC=00000020 | INSTR=40115293 | ALU=00000005 | WB=00000005
# Time=230000 | PC=00000024 | INSTR=0020a313 | ALU=00000000 | WB=00000000
# Time=250000 | PC=00000028 | INSTR=0060b393 | ALU=00000001 | WB=00000001
# Time=270000 | PC=0000002c | INSTR=002081b3 | ALU=0000000f | WB=0000000f
# Time=290000 | PC=00000030 | INSTR=40208233 | ALU=fffffffb | WB=fffffffb
# Time=310000 | PC=00000034 | INSTR=0020f2b3 | ALU=00000000 | WB=00000000
# Time=330000 | PC=00000038 | INSTR=0020e333 | ALU=0000000f | WB=0000000f
# Time=350000 | PC=0000003c | INSTR=0020c3b3 | ALU=0000000f | WB=0000000f
# Time=370000 | PC=00000040 | INSTR=00209433 | ALU=00001400 | WB=00001400
# Time=390000 | PC=00000044 | INSTR=0020d4b3 | ALU=00000000 | WB=00000000
# Time=410000 | PC=00000048 | INSTR=4020d533 | ALU=00000000 | WB=00000000
# Time=430000 | PC=0000004c | INSTR=0020a5b3 | ALU=00000001 | WB=00000001
# Time=450000 | PC=00000050 | INSTR=0020b633 | ALU=00000001 | WB=00000001
# Time=470000 | PC=00000054 | INSTR=00312023 | ALU=0000000a | WB=0000000a
# Time=490000 | PC=00000058 | INSTR=00012183 | ALU=0000000a | WB=0000000f
# Time=510000 | PC=0000005c | INSTR=00208463 | ALU=0000000f | WB=0000000f
# Time=530000 | PC=00000060 | INSTR=00100193 | ALU=00000001 | WB=00000001

```

```

# Time=550000 | PC=00000064 | INSTR=00109463 | ALU=0000000a | WB=0000000a
# Time=570000 | PC=00000068 | INSTR=00200193 | ALU=00000002 | WB=00000002
# Time=590000 | PC=0000006c | INSTR=0020c463 | ALU=0000000f | WB=0000000f
# Time=610000 | PC=00000074 | INSTR=0020d463 | ALU=0000000f | WB=0000000f
# Time=630000 | PC=00000078 | INSTR=00400193 | ALU=00000004 | WB=00000004
# Time=650000 | PC=0000007c | INSTR=fff00093 | ALU=ffffff | WB=ffffff
# Time=670000 | PC=00000080 | INSTR=00100113 | ALU=00000001 | WB=00000001
# Time=690000 | PC=00000084 | INSTR=0020e463 | ALU=00000000 | WB=00000000
# Time=710000 | PC=00000088 | INSTR=00100293 | ALU=00000001 | WB=00000001
# Time=730000 | PC=0000008c | INSTR=00200293 | ALU=00000002 | WB=00000002
# Time=750000 | PC=00000090 | INSTR=123450b7 | ALU=00001404 | WB=12345000
# Time=770000 | PC=00000094 | INSTR=004000ef | ALU=ffffffb | WB=00000098
# Time=790000 | PC=00000098 | INSTR=00500113 | ALU=00000005 | WB=00000005
# Time=810000 | PC=0000009c | INSTR=00001297 | ALU=00000000 | WB=0000109c
# Time=830000 | PC=000000a0 | INSTR=00010283 | ALU=00000005 | WB=00000000
# Time=850000 | PC=000000a4 | INSTR=00014303 | ALU=00000005 | WB=00000000
# Time=870000 | PC=000000a8 | INSTR=00011383 | ALU=00000005 | WB=00000000
# Time=890000 | PC=000000ac | INSTR=00015403 | ALU=00000005 | WB=00000000
# Time=910000 | PC=000000b0 | INSTR=87654337 | ALU=00000000 | WB=87654000
# Time=930000 | PC=000000b4 | INSTR=32130313 | ALU=87654321 | WB=87654321
# Time=950000 | PC=000000b8 | INSTR=00612423 | ALU=0000000d | WB=0000000d
# Time=970000 | PC=000000bc | INSTR=00814503 | ALU=0000000d | WB=00000043
# Time=990000 | PC=000000c0 | INSTR=00814583 | ALU=0000000d | WB=00000043
# Time=1010000 | PC=000000c4 | INSTR=00811603 | ALU=0000000d | WB=00006543
# Time=1030000 | PC=000000c8 | INSTR=00815683 | ALU=0000000d | WB=00006543
# Time=1050000 | PC=000000cc | INSTR=0000006f | ALU=00000000 | WB=000000d0

```

Simulation outputs confirmed correct execution for all supported instructions.

12.Design Decisions & Limitations

Intentional Simplifications:

- Single-cycle (no pipelining)
- No misaligned memory access handling
- No exceptions or interrupts
- No CSR instructions

These choices were made to keep the design clear, testable, and aligned with the educational goal of understanding CPU Fundamentals. These decisions reduce complexity and make behavior deterministic.

13.Conclusion

This Project demonstrates a fully functional RV32I compatible single-cycle CPU implemented from scratch in Verilog. It accurately executes a broad subset of RISC-V instructions and provides a strong foundation for more advanced designs such as pipelined or superscalar processors and closely follows the official RISC-V specification.

The modular structure allows future extensions such as pipelining, caches, or additional instruction support.

In addition, a software compiler could be written to target this CPU. Because the processor implements the RV32I base instruction set in a standards-compliant manner, it is suitable as a compilation target. A compiler could be developed to generate RV32I machine code for this CPU.

14.References

The RISC-V Instruction Set Manual, Volume 1: Base User-Level ISA 2011

Andrew Waterman, Yunsup Lee, David A. Patterson, Krste Asanovic